

Design Task

Description: The weather station sends the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; and the total rainfall; as sampled at five-minute intervals.

Stimulus: The weather information system establishes a link with the weather station and requests transmission of the data.

Response: The sensors data is sent to the weather information system.

Additional Comments: Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

The Design Decisions

Design Patterns			
Design Problem	Design Solution	Participants and their Application in Reference Model	Collaboration
<i>Event-Driven development</i>	<i>Observer Pattern [1] [2]</i>	Subject (Subject), ConcreteSubject (WeatherStation), Observer (Observer), ConcreteObserver (WeatherInformationSystem)	WeatherStation notifies WeatherInformationSystem whenever a change occurs that could make WeatherInformationSystem' state inconsistent with its own. After being informed of a change in the WeatherStation, a WeatherInformationSystem object may query the subject for information. WeatherInformationSystem uses this information to reconcile its state with that of the subject.
<i>Algorithm-driven concern</i>	<i>Strategy Pattern [2]</i>	Strategy (ReportingStrategy), ConcreteStrategy (HourlyReporting, CustomIntervalReporting), Context (ReportingContext)	ReportingStrategy and ReportingContext interact to implement the chosen algorithm. ReportingContext passes all data required by the algorithm to the strategy when the algorithm is called and forwards requests from WeatherStation to its strategy. WeatherStation interacts with the

		ReportingContext exclusively.
Design Principles		
Design Concern	Design Principle	Application in Reference Model
<i>Abstraction</i>	<i>DRY [1]</i>	Abstracting the common features of different sensors in parent class Sensor.

The Reference Model

The PlantUML Description
<pre> @startuml abstract class Subject { +attach(o: Observer) +detach(o: Observer) +notify() } note left of Subject::notify forall o in observers{ o->Update() } end note abstract class Observer { +{abstract} update() } Subject --> Observer : observers class WeatherStation extends Subject{ -stationId: String -location: String -maxAirTemp: float -minAirTemp: float -avgAirTemp: float -maxGroundTemp: float -minGroundTemp: float -avgGroundTemp: float -maxPressure: float -minPressure: float -avgPressure: float -maxWindSpeed: float -minWindSpeed: float -avgWindSpeed: float -totalRainfall: float +collectData(s: Sensor) +SetState() +GetState() +notify() } Class ReportingContext{ </pre>

```

+reportingInterval()
}
note right of ReportingContext::reportingInterval
    reportingStrategy->report()
end note
note left of WeatherStation::GetState
    return maxAirTemp,
    minAirTemp,
    avgAirTemp,
    maxGroundTemp,
    minGroundTemp,
    avgGroundTemp,
    maxPressure,
    minPressure,
    avgPressure,
    maxWindSpeed,
    minWindSpeed,
    avgWindSpeed,
    totalRainfall
end note
class WeatherInformationSystem extends Observer{
    -systemId: String
    -weatherState: List<float>
    +requestData()
    +update()
}
note left of WeatherInformationSystem::weatherState
    weatherState = subject->GetState()
end note
WeatherInformationSystem --> WeatherStation : subject

abstract class ReportingStrategy {
    +{abstract} report()
}

class HourlyReporting extends ReportingStrategy{
    +report()
}

class CustomIntervalReporting extends ReportingStrategy{
    -interval: int
    +report()
}

WeatherStation --> ReportingContext
ReportingContext --o ReportingStrategy : reportingStrategy

abstract class Sensor {
    -sensorId: String
    +readValue()
}

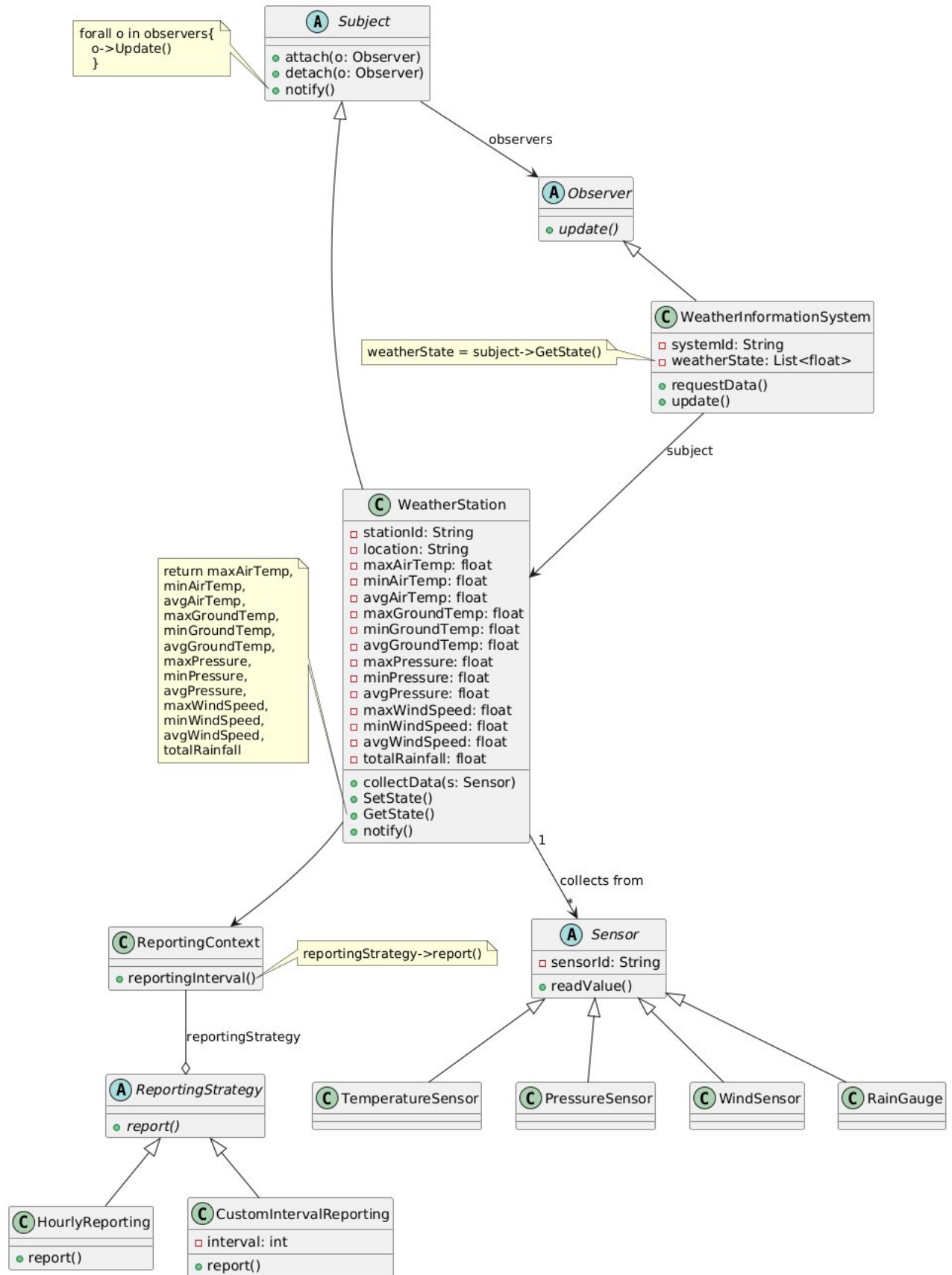
```

```
class TemperatureSensor extends Sensor
class PressureSensor extends Sensor
class WindSensor extends Sensor
class RainGauge extends Sensor

WeatherStation "1" --> "*" Sensor : collects from

@enduml
```

Visualization



References

1. Thomas, David, and Andrew Hunt. *The Pragmatic Programmer: your journey to mastery*. Addison-Wesley Professional, 2019.
2. Gamma, E., et al. "Design patterns: elements of reusable software." *Addison Welsey* (1994).